Lecture 6

A finite gate set: the circuit model

Objectives. *Lecture 5 counted queries to a hidden function. This lecture asks:* what is a quantum algorithm, and what can be proved about the power of such algorithms? *A quantum algorithm writes, for each input, a circuit over the alphabet* $\{H, S, T, \mathrm{CNOT}\}$ *and inverses; errors add (Theorem 6.2), so with Clifford+*T *universality and Solovay–Kitaev (cited) little depends on the alphabet. The main result is* $\mathsf{BPP} \subseteq \mathsf{BQP} \subseteq \mathsf{PSPACE}$ *(Theorems 6.11 and 6.12): a proof that* $\mathsf{BQP} \neq \mathsf{BPP}$ *would prove* $\mathsf{P} \neq \mathsf{PSPACE}$*. This is the course's first answer.*

By the end of this lecture you will be able to:

- Prove that errors add: replacing each gate U_t by V_t moves every outcome probability by at most $2\sum_t \|U_t - V_t\|$.
- State Clifford+T universality and Solovay–Kitaev with their hypotheses, and show that a global phase becomes relative under a control.
- Define BQP and BPP as decision classes, and amplify a success probability of $\frac{2}{3}$ to $1 - \delta$ with $O(\log(1/\delta))$ runs.
- Prove $\mathsf{BPP} \subseteq \mathsf{BQP}$, sketch $\mathsf{BQP} \subseteq \mathsf{PSPACE}$ from the path sum, and say why a proof of $\mathsf{BQP} \neq \mathsf{BPP}$ would prove $\mathsf{P} \neq \mathsf{PSPACE}$.

6.1 Errors add

No gate is built exactly, and § 6.2 will replace gates by words in a few fixed ones, exact up to an error. How do such errors add up in a long circuit?

Definition 6.1 (The distance of gates)**.** The *norm* of a matrix A is

$$\|A\| = \max_{\|\psi\|=1} \|A\psi\|,$$

and the *distance* of two gates U and V of one size is $\|U - V\|$.

On a register a gate acts as $U \otimes \mathbb{1}$, up to a reordering of the wires, which is a unitary permutation. For $\psi = \sum_j \psi_j \otimes |j\rangle$, (3.1) gives $\|(A \otimes \mathbb{1})\psi\|^2 = \sum_j \|A\psi_j\|^2 \leq \|A\|^2 \|\psi\|^2$. So gates on the same wires are no farther apart on the whole register than on their own wires.

Theorem 6.2 (Errors add)**.** *Let* $U_1, \ldots, U_L$ *and* $V_1, \ldots, V_L$ *be unitaries of one size, with* $U = U_L \cdots U_1$, $V = V_L \cdots V_1$ *and* $\delta = \sum_t \|U_t - V_t\|$.

(i) $\|U - V\| \leq \delta$.

(ii) For every state ψ *and every projector* Π ($\Pi^2 = \Pi = \Pi^\dagger$),

$$\left| \|\Pi U\psi\|^2 - \|\Pi V\psi\|^2 \right| \leq 2\delta.$$

Every outcome of a projective measurement (Definition 3.4), and every set of outcomes, has a probability $\|\Pi\varphi\|^2$, so (ii) bounds them all.

Proof. (i) With the unitaries $W_t = U_L \cdots U_{t+1}$ and $W'_t = V_{t-1} \cdots V_1$, the sum

$$U - V = \sum_{t=1}^{L} W_t (U_t - V_t) W'_t$$

telescopes, as expanding each bracket shows. A unitary keeps lengths and maps the unit vectors onto themselves, so each term has the norm $\|U_t - V_t\|$, and the triangle inequality gives (i). (ii) The states $\varphi = U\psi$ and $\chi = V\psi$ have $\|\varphi - \chi\| \le \delta$ by (i). A projector does not lengthen a vector, since $\|\Pi v\|^2 + \|(\mathbb{1} - \Pi)v\|^2 = \|v\|^2$. So

$$\begin{aligned} &\left| \|\Pi\varphi\|^2 - \|\Pi\chi\|^2 \right| \\ &\quad = \left| \|\Pi\varphi\| - \|\Pi\chi\| \right| \left(\|\Pi\varphi\| + \|\Pi\chi\| \right) \\ &\quad \le \|\Pi(\varphi - \chi)\| \cdot 2 \le 2\delta. \end{aligned}$$

■

So L gates, each within $\epsilon/(2L)$ of its target, move no probability by more than ϵ: errors add, and do not multiply. They can really add.

Example 6.3 (Q: errors that line up)**.** Let $V = \mathrm{diag}(1, e^{i(\pi/4+\epsilon)})$ be a T gate whose phase is off by ϵ. The norm of a diagonal matrix is the largest modulus of its entries, so $\|T - V\| = \left|1 - e^{i\epsilon}\right|$. Eight in a row, $T^8 = \mathbb{1}$ and $V^8 = \mathrm{diag}(1, e^{8i\epsilon})$, so

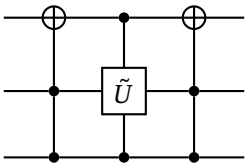
$$\left\| T^8 - V^8 \right\| = \left| 1 - e^{8i\epsilon} \right|,$$

against the bound $8\left|1 - e^{i\epsilon}\right|$ of (i): for $\epsilon = 0.01$, 0.07998 against 0.08000. Errors that do not cancel reach the bound, to first order in ϵ.

6.2 Exact and Clifford+T universality, Solovay–Kitaev

Theorem 6.4 (Exact universality · *cited*)**.** *Every unitary on n qubits is a product of $O(n^2 4^n)$ CNOTs and one-qubit gates.*

The proof (Nielsen and Chuang, §§4.5.1–4.5.2, pp. 189–193) writes the unitary as a product of at most $2^{n-1}(2^n - 1)$ *two-level* unitaries, each acting as a 2×2 unitary $\tilde{U}$ on two basis vectors and as $\mathbb{1}$ on the others. A two-level unitary is a permutation of the basis that makes its two vectors differ in one bit, a controlled $\tilde{U}$, and the permutation undone. For $|101\rangle$ and $|011\rangle$:



The Toffoli sends 011 to 111 and fixes 101; $\tilde{U}$ acts on wire 2 when wires 1 and 3 hold 1, so on $|101\rangle$ and $|111\rangle$. The circuit acts as $\tilde{U}$ on $|101\rangle$, $|011\rangle$, in this order, and as $\mathbb{1}$ on the other six. Gates with several controls are products of CNOTs and one-qubit gates (§4.3).

Theorem 6.5 (Clifford+T universality · *cited*)**.** *For every one-qubit gate U and every $\epsilon > 0$ there are a product V of H and T gates and a real α with $\|U - e^{i\alpha}V\| < \epsilon$.*

Nielsen and Chuang prove it (§4.5.3, pp. 194–197), citing one step (p. 214).

The phase cannot be dropped: $\det H = -1$, $\det S = i$ and $\det T = e^{i\pi/4}$, so every product of the alphabet's one-qubit gates has an eighth root of unity as its determinant, and by continuity none comes close to $e^{i\pi/16}\mathbb{1}$, of determinant $e^{i\pi/8}$. No reading sees a global phase (Theorem 3.7), and the phases of gates multiply into one: if $\|U_t - e^{i\alpha_t}V_t\| \le \delta_t$ for each t, then Theorem 6.2 for the gates $e^{i\alpha_t}V_t$, whose product is $e^{i\alpha}V$ with $\alpha = \sum_t \alpha_t$, puts every probability of V within $2\sum_t \delta_t$ of U's.

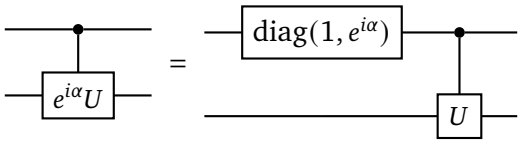
Theorem 6.6 (Solovay–Kitaev · *cited*)**.** *Let $\mathcal{G}$ be a finite subset of $\mathrm{SU}(d)$, the $d \times d$ unitaries of determinant 1, that contains the inverse of each of its elements and whose products approximate every element of $\mathrm{SU}(d)$ arbitrarily well. There is a constant c such that for every $U \in \mathrm{SU}(d)$ and $\epsilon > 0$ a product W of $O(\log^c(1/\epsilon))$ elements of $\mathcal{G}$ has $\|U - W\| < \epsilon$.*

This is Dawson and Nielsen's Theorem 1 (p. 4); Nielsen and Chuang prove the case $d = 2$ (p. 618), where the gates H, S and T, each times a phase that puts it in $\mathrm{SU}(2)$, form with their inverses such a set. For it, a classical algorithm, given U, finds W in time polynomial in $\log(1/\epsilon)$ (p. 4; Kitaev, Shen and Vyalyi, pp. 76–78). The inverses are a hypothesis, which is why the alphabet contains them.

6.3 The control-phase guard

A gate's global phase changes nothing (Theorem 3.7). Under a control it does.

Example 6.7 (QQ: a global phase under a control)**.** By the interchange law, $\mathrm{C}(e^{i\alpha}U) = |0\rangle\langle 0| \otimes \mathbb{1} + e^{i\alpha}|1\rangle\langle 1| \otimes U$ is



(6.1)

an equality of matrices. The global phase of U has become a relative phase of the control. For $U = \mathbb{1}$ and $\alpha = \pi$: $\mathrm{C}(\mathbb{1}) = \mathbb{1}$, while $\mathrm{C}(-\mathbb{1}) = Z \otimes \mathbb{1}$; with the control in $|+\rangle$, H and a reading of the control give 0 in the first case and 1 in the second, with certainty.

So a gate compiled up to a global phase may not be put under a control: to control U, compile $\mathrm{C}(U)$ itself, a gate on one more qubit. Lecture 7 controls powers of a gate whose eigenphase is the answer.

6.4 The circuit model, BQP, BPP, amplification

A quantum algorithm handles inputs of every length, so it is a rule that writes a circuit for each. The rule is a classical algorithm, which we take as known (a program, without coins); it runs in *polynomial time* if, for some polynomial p, it stops after at most $p(n)$ steps on every input of length n. A *decision problem* is a function $a\colon \{0,1\}^* \to \{0,1\}$ on all strings; $a(x)$ is the answer on the input x.

Definition 6.8 (The circuit model; BQP, BPP)**.** The *alphabet* is $\{H, S, T, \text{CNOT}\}$ with the inverses $S^\dagger$ and $T^\dagger$.

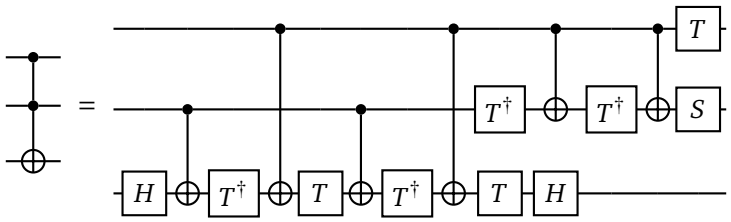
(i) A decision problem a is in BQP if a classical algorithm, given x, writes in polynomial time a circuit C_x over the alphabet such that C_x, run on $|0\cdots0\rangle$ with its top wire read, gives $a(x)$ with probability at least $\frac{2}{3}$.

(ii) It is in BPP if a classical algorithm, given x, writes in polynomial time a circuit D_x of AND, NOT and FAN gates (Definition 1.1) with k inputs and one output, such that $D_x(r) = a(x)$ for at least $\frac{2}{3}$ of the strings $r \in \{0,1\}^k$, its *coins*.

(iii) It is in P if a classical algorithm computes $a(x)$ in polynomial time, and in PSPACE if one computes it with memory polynomial in the length of x, in any time.

This is the usual BPP: a polynomial-time algorithm that tosses coins, run on x and written out step by step, is such a circuit in its coins (Kitaev, Shen and Vyalyi, Def. 4.2, Thm 2.3). The *alphabet* is finite on purpose: if the algorithm could name any gate with computable entries, the gate equal to X when $a(x) = 1$ and to $\mathbb{1}$ otherwise would decide a alone, the work hidden in its entries.

Classical gates are exact. CNOT is in the alphabet, $X = HS^2H$, and the Toffoli is (Nielsen and Chuang, Fig. 4.9)



(6.2)

an equality of matrices.

The threshold. The constant $\frac{2}{3}$ is de Wolf's (p. 110); Nielsen and Chuang use $\frac{3}{4}$ (p. 201). Any constant strictly between $\frac{1}{2}$ and 1 gives the same class:

Proposition 6.9 (Amplification)**.** *Let a procedure be wrong with probability at most $q < \frac{1}{2}$. The majority answer of k independent runs of it, a tie counted as wrong, is wrong with probability at most $(4q(1-q))^{k/2}$. For $q = \frac{1}{3}$ this is $(8/9)^{k/2}$, at most δ once $k \geq 17\ln(1/\delta)$.*

Proof. Let a run be wrong with probability $q' \leq q$. The majority is wrong only if some set of $j \geq k/2$ runs is wrong and the others right, which has the probability

$$q'^{\,j}(1-q')^{k-j} = \big(q'(1-q')\big)^{k/2}\,\rho^{\,j-k/2} \leq \big(q(1-q)\big)^{k/2},$$

with $\rho = q'/(1-q') \leq 1$, since $x(1-x)$ increases on $[0, \frac{1}{2}]$. There are at most 2^k such sets. And $(8/9)^{k/2} \leq \delta$ when $k \geq 2\ln(1/\delta)/\ln(9/8)$, and $2/\ln(9/8) < 17$. ∎

The vote runs inside the model: put k copies of C_x side by side and compute the majority of their top wires into a new top wire by Theorem 1.9, which uses them only as controls. By Theorem 4.5, reading the new wire gives what reading the k wires and voting gives. For a fixed δ it takes polynomial time.

Other gates, as in Lecture 7, are compiled:

Corollary 6.10 (Compiling · *cited*)**.** *Fix k. For a circuit of L gates, each on at most k qubits, and $\epsilon > 0$, a circuit over the alphabet of $O(L\log^c(L/\epsilon))$ gates gives every outcome on every input state with a probability within ϵ of the given circuit's. Given the gates' entries to $O(\log(L/\epsilon))$ bits, a classical algorithm writes it in time polynomial in L and $\log(1/\epsilon)$.*

Proof. By Theorem 6.4 each gate is a product of at most K CNOTs and one-qubit gates, K depending on k only. A one-qubit gate is a phase times an element of SU(2), so Theorem 6.6 gives, for each, a word in H, S, T and inverses within $\epsilon/(2KL)$ of it up to a phase, of length $O(\log^c(2KL/\epsilon))$. The phases multiply into one, and no probability moves by more than ϵ (§ 6.2). ∎

So allowing such gates does not enlarge BQP, if their entries can be computed to b bits in time polynomial in b and the length of x: amplify to success $\frac{11}{12}$, then compile with $\epsilon = \frac{1}{12}$, and the success is at least $\frac{5}{6}$.

6.5 BPP ⊆ BQP ⊆ PSPACE

Theorem 6.11 (BPP ⊆ BQP)**.** P ⊆ BPP ⊆ BQP.

Proof. P ⊆ BPP: D_x may ignore its coin and output $a(x)$. Let $a \in$ BPP, with D_x of size s and k coins. Theorem 1.9 gives a circuit R_x of at most $4s + 1$ Toffoli, CNOT and NOT gates that maps $(r, y, 0)$ to $(r, y \oplus D_x(r), 0)$; draw y as the top wire. Write each NOT as HS^2H and each Toffoli by (6.2), and put H on each coin wire first: this is C_x, over the alphabet and written in polynomial time. By (5.2) the Hadamards make $2^{-k/2}\sum_r |r\rangle$ on the coin wires, and by linearity

$$C_x\,|0\cdots0\rangle = 2^{-k/2}\sum_r |D_x(r)\rangle\,|r\rangle\,|0\rangle\,.$$

By (3.2), the top wire reads $a(x)$ with probability 2^{-k} times the number of r with $D_x(r) = a(x)$, which is at least $\frac{2}{3}$. ∎

Theorem 6.12 (BQP ⊆ PSPACE · *sketched*)**.** BQP ⊆ PSPACE.

Sketch. Let $a \in$ BQP, and let C_x have L gates on m wires, h of them Hadamards; if $L = 0$, the top wire reads 0. Cut C_x after every gate, and use the path sum (Theorem 2.3) from $|0\cdots0\rangle$. Each entry of H is $\pm2^{-1/2}$, while S, T, their inverses and CNOT have in each column one non-zero entry, a power of $\zeta = e^{i\pi/4}$. So a path of non-zero amplitude is fixed by its bits $c \in \{0,1\}^h$ after the Hadamards, and its amplitude is $2^{-h/2}\zeta^{j(c)}$ for an integer $j(c)$; let $y(c)$ be its end. The top wire reads 1 with probability

$$\begin{aligned} p &= \textstyle\sum_{y_1=1}\left|\sum_{y(c)=y} 2^{-h/2}\zeta^{j(c)}\right|^2 \\ &= 2^{-h}\textstyle\sum_{c,c'}\zeta^{j(c)-j(c')}, \end{aligned}$$

the last sum over the pairs with $y(c) = y(c')$ and first bit 1. Let M_d count the pairs with $j(c) - j(c') \equiv d$ modulo 8. Exchanging c and c' gives $M_d = M_{8-d}$, so

$$p = 2^{-h}\big((M_0 - M_4) + \sqrt{2}(M_1 - M_3)\big).$$

An algorithm goes through the 4^h pairs one at a time, following both paths. Besides C_x it holds two strings of m bits, two exponents, and eight counters of at most $2h + 1$ bits. Then $a(x) = 1$ exactly when $p > \frac{1}{2}$, a comparison of integers and $\sqrt{2}$. The memory is polynomial, the time exponential in h. ∎

With Theorem 6.11,

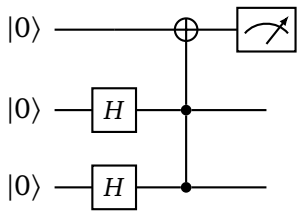
$$\text{P} \subseteq \text{BPP} \subseteq \text{BQP} \subseteq \text{PSPACE},$$

each inclusion proved. A proof that BQP ≠ BPP would give BPP ≠ PSPACE (Nielsen and Chuang, p. 201), and so P ≠ PSPACE (de Wolf, pp. 111–112), open since the 1960s. That BQP ≠ BPP is a conjecture; Lecture 8 gives its main evidence.

6.6 The coin

A probabilistic algorithm tosses its coins as it goes. The proof of Theorem 6.11 runs it unchanged, and "unchanged" hides two steps.

Example 6.13 (Q: a coin made of a Hadamard). H on $|0\rangle$, read, is a fair coin (Lecture 2). Let two coins feed one AND gate, whose output is 1 with probability $\frac{1}{4}$. Its reversible form is a Toffoli onto a fresh wire (Example 1.10), and the proof's circuit becomes



which makes $\frac{1}{2}\sum_r |\mathrm{AND}(r_1,r_2)\rangle\,|r_1\rangle\,|r_2\rangle$: the top wire reads 1 with probability $\frac{1}{4}$.

The first step: the classical gates become Toffoli, CNOT and NOT (Theorem 1.9), exact over the alphabet by (6.2). The second: the classical run reads each coin when it is tossed, and later gates use the outcome; here, read wires 2 and 3 after the Hadamards, then apply X to the top wire if both read 1. The Toffoli uses wires 2 and 3 only as controls, so by Theorem 4.5 the readings move to the end with the same joint statistics, the use Example 4.6 announced. When the coins are read, or whether they are, changes nothing in the answer's statistics.

6.7 The first answer

The course asks what a quantum computation is, and what makes the difference visible. Its first answer: *a quantum computation is a family of circuits over a finite universal gate set, written by a classical polynomial-time algorithm, run on* $|0\cdots0\rangle$ *and measured.* BQP is the class of decision problems such computations solve with bounded error, and it is robust: neither the threshold, strictly between $\frac{1}{2}$ and 1, nor gate errors with a small sum, nor other gates on a few qubits with efficiently computable entries change it (§§6.1, 6.4).

What is proved: BPP $\subseteq$ BQP $\subseteq$ PSPACE. The 2^m amplitudes of a register do not take a computation beyond PSPACE: the path sum adds them up in polynomial memory. Whether they take it beyond BPP is open; Lecture 5's separation holds relative to an oracle. The next lectures look for the difference: Lecture 8 a problem, factoring, that a quantum computer solves in polynomial time with bounded error and no known classical algorithm does, and Lecture 9 a query problem with a proved bound on each side.

Summary.

- *Errors add:* every outcome probability moves by at most $2\sum_t \|U_t - V_t\|$ (Theorem 6.2).
- *Universality* (cited): H and T approximate every one-qubit gate up to a global phase, with $O(\log^c(1/\epsilon))$ gates (Theorems 6.5 and 6.6); under a control the phase counts (Example 6.7).
- *The model:* BQP and BPP are decision classes, with circuits written in polynomial time; a vote amplifies $\frac{2}{3}$ to $1-\delta$ (Definition 6.8 and Proposition 6.9).
- *The main result:* BPP $\subseteq$ BQP $\subseteq$ PSPACE (Theorems 6.11 and 6.12).

- *Next:* Lecture 7, Fourier sampling and phase estimation.

References

Nielsen and Chuang, §4.3, pp. 180–184, §4.5, pp. 189–197 and 200–202, and App. 3, pp. 617–619. De Wolf, pp. 15–16, 110–113 and 200–201. Kitaev, Shen and Vyalyi, pp. 22, 36–37, 83 and 91. Watrous, *Quantum computational complexity* (2008), pp. 12–13. C. M. Dawson and M. A. Nielsen, *The Solovay–Kitaev algorithm*, arXiv:quant-ph/0505030 (2005), pp. 2–4.